Chapter 6: Conditional Processing

Conditional Branching:
    A programming language that permits decision making lets you alter the flow of control
    using a technique known as "Conditional Branching"

Boolean Instructions:
    - AND || and operation between a source operand and a destination operand
    - OR || or operation between a source operand and a destination operand
    - XOR || exclusive or operation between a source operand and a destination operand
    - NOT || not operation on the destination operand
    - TEST || Implied and operation between a source and destination operand, setting The
              CPU flags appopriately

    CPU Status Flags:
        - boolean intructions affect zero, carry, sign, overflow and parity flags.
        - The zero flag is set when the result of an operation equals to 0.
        - The carry flag is set when the operation generates a carry out of the highest bit of
        the destination operand
        - The sign flag is the copy of the high bit of the destination operand, indicating that
        it is negative if it is set and positive if clear.
        - The overflow flag is set when an instruction generates a result that is outside the
        signed range of the destination operand
        - The parity flag is set when an instruction generates an even number of 1 bits in the low
        byte of the destination operand

    Syntax for AND:
        and destination, source
        Allowed Synatax:
            and register, register
            and register, mem
            and register, imm
            and mem, register
            and mem, imm
            "Operand must be of the same size"
            The and instruction always clears the carry and the overflow flags. It modifies the
            sign, zero, and the parity flags in a way that is consistent with the value assigned
            to the destination operand

    Syntax for OR:
    or destination, source
        Allowed Syntax:
            or register, register

```
            or register, mem
            or register, imm
            or mem, register
            or mem, imm
            "Operand must be of the same size"
            The or instruction always clears the carry and the
overflow flags. It modifies the sign,
            zero, and the parity flags in a way that is
consistent with the value assigned to the
            destination operand

    Syntax for XOR:
        xor destination, source
        The xor flag always clears the overflow and the carry
flags. It modifies the sign, zero and
        parity flags in a way that is consistent with the
value assigned to the destination operand.

    Syntax for NOT:
        not destination
        Allowed Syntax:
            not register
            not mem

        the not instruction inverts all bits in an
operand[one's complement]
        no flags are affected by the not instruction

    TEST Instruction:
        - performs an implied and operation between
corresponding bits in the two operands
          and subroutines the flags without modifying either
operand.

        - It sets the sign, zero, and parity flags based on
the value assigned to the destination
          operand

        - The test instruction permits the same operand
combinations as the and instruction

        - test is particularly valuable for finding out
whether individuals bits in an operand
          are set.

Comparison Instruction:

        - Assembly uses cmp instruction to compare integers

          Syntax:
                cmp destination, source

          Allowed Syntax:
                cmp register, register
                cmp register, mem
                cmp register, imm
                cmp mem, register
```

cmp mem, imm

Working:
* if source is greater than destination then carry flag is set
and zero flag is cleared

* if destination is greater than source then both the flags are cleared

* is destination is equals to source then zero flag is set and carry
flag is cleared

Unsigned:
* if source is greater than destination then sign and overflow flags are not equal

* if destination is greater than source then sign and overflow flags are equal.

* is destination is equals to source then zero flag is set

- to clear the zero flag the register can be OR'ed with 1

- stc sets the carry flag

- clc clears the carry flag

- to clear the overflow flag the register can be OR'ed with 0

- incrementing the register when its already storing the maximum possible value will
set the overflow flag

Conditional Jumps:

- JC = jump if carry flag is set
- JNC = jump if carry flag is cleared
- JZ = jump if zero flag is set
- JNZ = jump if zero flag is cleared
- JO = jump if overflow flag is set
- JNO = jump if overflow flag is cleared
- JS = jump if signed flag is set
- JNS = jump if signed flag is cleared
- JP = jump if parity flag is even
- JNP = jump if parity flag is odd
- JE = jump if leftoperand is equal to rightoperand
- JNE = jump if leftoperant is not equal to rightoperand
- JCX = jump if CX = 0

Unsigned Conditional Jumps:

- JA = jump if leftoperand > rightoperand

- JNBE = same as JA
- JAE = jump if leftoperand >= rightoperand
- JNB = same as JAE
- JB   = jump if leftoperand < rightoperand
- JNAE = same as JB
- JBE = jump if leftoperand <= rightoperand
- JNA = same as JBE

signed Conditional Jumps:

- JG   = jump if leftoperand > rightoperand
- JNLE = same as JG
- JGE = jump if leftoperand >= rightoperand
- JNL = same as JGE
- JL   = jump if leftoperand < rightoperand
- JNGE = same as JL
- JLE = jump if less than or equals to leftoperand <= rightoperand
- JNG = same as JLE

Conditional Loop Instructions:

- Loopz Instruction:
  the loopz instruction works just like the loop instruction except that it has one
  additional condition: the zero flag must be set in order for control to transfer to
  the destination label

- loope instruction is same as loopz

- loopnz Instruction:
  the loopnz instruction is the counterpart of loopz. The loop continues while
  the unsigned value of ecx is greater than zero and the zero flag is clear

- loopne instruction is same as loopnz

- While Loop:
  the while loop in assembly language is just the synatx difference from the original loop
  Syntax:

```
top: cmp eax, ebx
     jae next
     inc eax
     jmp top
next:
```